

Flutter Development Environment Setup (2026)

Dr M AOUE

Visual Studio Code Edition

Context. This guide sets up Flutter for students who prefer **Visual Studio Code** to Android Studio. VS Code is lighter, and the Flutter extension gives you the same daily tools: hot reload, debugging, DevTools, and refactoring. The steps work on Windows, macOS, and Linux. The Android Studio edition of this guide (001_install) stays valid: choose the editor you like better.

Not the same product. *Visual Studio Code* (the editor you install here) is different from *Visual Studio* (the big Windows IDE). You only need Visual Studio if you want to build *Windows desktop* apps.

Quick Start (10-minute setup)

1. Install **Git** and **Visual Studio Code**
2. In VS Code, install the **Flutter** extension (Dart-Code.flutter; it also installs Dart)
3. Press **Ctrl+Shift+P** → **Flutter: New Project** → **Download SDK** → **Clone Flutter** → **Add SDK to PATH**
4. Close all terminals, restart VS Code, and run `flutter doctor -v`
5. **Flutter: New Project** → **Application**, pick **Chrome** in the status bar, press **F5**

1 Prerequisites

One consistent setup for Windows, macOS, and Linux that avoids version drift and gives you hot reload, debugging, testing, profiling, and CI from day one, without Android Studio.

Requirement	Version / Note
OS	Windows 10/11, macOS 13.5+, Ubuntu 22.04+
RAM	16 GB minimum (32 GB recommended if you also run the Android emulator)
Storage	50+ GB SSD free (SDKs, emulators, caches)
Editor	Visual Studio Code, latest stable
Git	2.34+ (needed for Flutter version management)
Internet	Required for SDK downloads and dependencies

Which targets do I need to set up? You can run the same Flutter app on several targets. Set up only what you need today.

Target	Extra setup	Good for
Chrome (web)	None: Chrome is enough	Fastest first run, no emulator
Desktop	Windows: Visual Studio with the “Desktop development with C++” workload. macOS: Xcode command-line tools. Linux: the GTK/clang toolchain from Flutter’s Linux setup page	Quick tests on your own machine
Android emulator or phone	Android SDK and a JDK (see Section 6)	Mobile work in this course

2 Install Git

```
1 git --version
```

Listing 1: Check Git

If missing:

Windows (PowerShell):

```
1 winget install --id Git.Git -e
```

macOS:

```
1 xcode-select --install # includes Git
2 # or:
3 brew install git
```

Ubuntu (this also installs the packages Flutter needs on Linux):

```
1 sudo apt-get update -y && sudo apt-get install -y curl git unzip xz-utils zip
  libglu1-mesa
```

Basic config:

```
1 git config --global user.name "Your Name"
2 git config --global user.email "you@example.com"
3 git config --global init.defaultBranch main
```

3 Install Visual Studio Code

Download it from <https://code.visualstudio.com/>, or from the command line:

Windows (PowerShell):

```
1 winget install --id Microsoft.VisualStudioCode -e
```

macOS:

```
1 brew install --cask visual-studio-code
```

Ubuntu:

```
1 sudo snap install code --classic
```

4 Install the Flutter Extension and SDK

1. Open VS Code and install the **Flutter** extension from the Marketplace (`Dart-Code.flutter`). It installs the Dart extension too.
2. Open the Command Palette (`Ctrl+Shift+P`, or `Cmd+Shift+P` on macOS) and run **Flutter: New Project**.
3. Choose **Download SDK** when VS Code asks for a Flutter SDK.
4. Pick an installation folder *without spaces or special characters*, for example `C:\dev` or `~/dev`.
5. Click **Clone Flutter**, wait for the download, then click **Add SDK to PATH**.
6. **Close all terminal windows and restart VS Code**, so the new PATH is used.

Verify (VS Code → Terminal → New Terminal):

```
1 flutter --version
2 flutter doctor -v
```

Warning: Choose installation paths without spaces or special characters to avoid build issues.

Alternative: install the SDK yourself

If you prefer a manual install (the same as the Android Studio edition), clone Flutter and tell VS Code where it is.

macOS/Linux

```
1 mkdir -p ~/dev && cd ~/dev
2 git clone https://github.com/flutter/flutter.git -b stable
3 echo 'export PATH="$HOME/dev/flutter/bin:$PATH"' >> ~/.zshrc
4 source ~/.zshrc
```

Windows (PowerShell)

```
1 mkdir C:\dev; cd C:\dev
2 git clone https://github.com/flutter/flutter.git -b stable
3 [Environment]::SetEnvironmentVariable(
4 "Path",
5 "C:\dev\flutter\bin;" + [Environment]::GetEnvironmentVariable("Path","User"),
6 "User"
7 )
```

Then open VS Code → Settings → Extensions → Dart, or edit `settings.json`:

```
1 {
2   "dart.flutterSdkPath": "C:\\dev\\flutter"
3 }
```

5 Editor Settings Worth Turning On

Open the Command Palette → **Preferences: Open User Settings (JSON)** and add:

```
1 {
2   "editor.formatOnSave": true,
3   "dart.previewFlutterUiGuides": true,
4   "dart.hotReloadOnSave": "manual"
5 }
```

With these settings, saving a file formats it, and (with an app running in debug mode) hot-reloads it.

6 Set Up Android (only if you target Android)

Skip this section if you only run on Chrome or desktop. For Android, Flutter needs the **Android SDK** and a **JDK**. Choose one of two ways to get them.

Option A (easiest): Android Studio as an SDK manager only

Install Android Studio once and use it *only* to download the SDK and create emulators. You still write code in VS Code, and Flutter finds Studio's embedded JDK automatically.

1. Install Android Studio from <https://developer.android.com/studio>.
2. Open **More Actions** → **SDK Manager**. Under **SDK Platforms**, install Android 16 (API 36) or newer.
3. Under **SDK Tools**, install: Android SDK Build-Tools, **Command-line Tools (latest)**, Android Emulator, Platform-Tools, CMake, and NDK (Side by side).
4. Create an emulator in **Device Manager** (a Pixel profile, API 36, with the system image that matches your CPU).
5. Accept the licenses: `flutter doctor --android-licenses`.

Option B: command-line SDK, no Android Studio

Use this if you want to avoid Android Studio completely. The example uses Windows paths.

1. **Install a JDK** (17 or 21). There is no embedded JDK without Android Studio:

```
1 winget install --id Microsoft.OpenJDK.21 -e --source winget
2 # Ubuntu: sudo apt install openjdk-17-jdk
```

2. Download **Command line tools only** from <https://developer.android.com/studio> and unzip so that `sdkmanager` ends up in `C:\Android\Sdk\cmdline-tools\latest\bin`.
3. Install the SDK packages (run `sdkmanager --list` to see the exact latest versions):

```
1 cd C:\Android\Sdk\cmdline-tools\latest\bin
2 $sdk = "C:\Android\Sdk"
3 $pkgs = "platform-tools","emulator","platforms;android-36",
4         "system-images;android-36;google_apis;x86_64"
5 .\sdkmanager.bat --sdk_root=$sdk $pkgs
```

Also install the newest `build-tools`;... shown by `sdkmanager --list`. CMake and NDK can be added the same way.

4. Tell Flutter where the SDK and JDK are, and accept the licenses:

```
1 flutter config --android-sdk C:\Android\Sdk
2 flutter config --jdk-dir "C:\Program Files\Microsoft\jdk-21.0.12-hotspot"
3 flutter doctor --android-licenses
```

Use the real JDK folder on your machine in the second command.

5. Create and start an emulator:

```
1 flutter emulators --create --name pixel_api36
2 flutter emulators --launch pixel_api36
```

On Apple Silicon Macs use an `arm64-v8a` system image instead of `x86_64`.

Real phone (both options)

Enable **Developer options** and **USB debugging** on the phone, connect it by USB, accept the prompt on the phone, then run `flutter devices`. On Windows you may need the phone maker's USB driver.

7 Verify Setup

From *VS Code* → *Terminal*:

```
1 flutter doctor -v
2 flutter devices
```

You should see green checks for Flutter and for each target you set up (Chrome, Android, or desktop), and at least one device. A warning for a platform you do not use, for example Visual Studio when you only build for Android and web, can be ignored.

Common fixes

- `flutter` not found → close all terminals and restart VS Code (or re-add Flutter's `bin` to `PATH`).
- VS Code cannot find the SDK → set `dart.flutterSdkPath` (Section 4).
- Licenses unknown → run `flutter doctor --android-licenses` again (as admin if needed).
- No devices → start an emulator, or connect a phone with USB debugging, or pick **Chrome**.

8 Create & Run a Test App

1. Command Palette → **Flutter: New Project** → **Application** → choose a folder and a name (for example `my_app`).
2. Click the device name at the bottom right of the status bar and choose **Chrome**, an emulator, or your phone.
3. Press F5 to run with the debugger, or `Ctrl+F5` to run without it.

The same thing from the terminal:

```
1 flutter create --platforms=android,ios,web my_app
2 cd my_app
3 code .
4 flutter run -d chrome
```

On macOS, first run Command Palette → **Shell Command: Install 'code' command in PATH** so that `code .` works.

Daily shortcuts

- **Hot reload**: the lightning-bolt button in the debug toolbar, or save the file (`Ctrl+S`) with the setting from Section 5.
- **Hot restart**: the circular-arrow button, or `Ctrl+Shift+F5`.
- **DevTools**: Command Palette → **Flutter: Open DevTools**.
- **Flutter doctor**: Command Palette → **Flutter: Run Flutter Doctor**.

9 Team Consistency (version pinning)

Create `.flutter-version` in your repo (example):

```
1 3.47.0
```

macOS/Linux bootstrap

```
1 #!/bin/bash
2 if [ -f ".flutter-version" ]; then
3 v=$(cat .flutter-version)
4 pushd ~/dev/flutter >/dev/null
5 git fetch --tags
6 git checkout "tags/$v"
7 popd >/dev/null
8 echo "Flutter locked to $v"
9 fi
```

Windows PowerShell bootstrap

```
1 if (Test-Path .flutter-version) {
2     $v = Get-Content .flutter-version
3     Push-Location "C:\dev\flutter"
4     git fetch --tags
5     git checkout "tags/$v" --quiet
6     Pop-Location
7     Write-Host "Flutter locked to $v"
8 }
```

Alt: teams often use FVM; this `.flutter-version` approach stays tool-agnostic and CI-friendly. These scripts assume you cloned Flutter into `~/dev/flutter` (macOS/Linux) or `C:\dev\flutter` (Windows). If VS Code downloaded the SDK to another folder, use that folder instead.

Final Validation Checklist

- ☐ `git --version & flutter --version` OK
- ☐ `flutter doctor -v` is green for the targets you use
- ☐ VS Code has the **Flutter** and **Dart** extensions
- ☐ A device shows in the VS Code status bar (Chrome, emulator, or phone)
- ☐ Android only: SDK **Command-line Tools** installed, JDK found, licenses accepted
- ☐ F5 runs the app; hot reload works

Troubleshooting Quick Reference

Symptom	Cause	Fix
flutter not found	PATH not refreshed	Close all terminals, restart VS Code; re-add Flutter bin to PATH if needed
“Unable to find Flutter SDK” in VS Code	SDK not registered	Set <code>dart.flutterSdkPath</code> in <code>settings.json</code>
Status bar shows “No Devices”	No emulator or phone running	Start an emulator, connect a phone, or use Chrome
cmdline-tools missing	Incomplete Android SDK	Install “Command-line Tools (latest)” with the SDK Manager or <code>sdkmanager</code>
No JDK found (Android toolchain)	No Android Studio, or no JDK on PATH	Install JDK 17 or 21, then run <code>flutter config --jdk-dir</code> with the JDK folder (Section 6)
Visual Studio missing components	Windows desktop workload not installed	Only needed for Windows desktop apps: add the “Desktop development with C++” workload, or ignore it for Android/web
Emulator slow	Missing hardware acceleration	Enable HVF/WHPX/KVM; use x86_64 images (arm64 on Apple Silicon)
iOS precache fails	Platform mismatch	Run <code>flutter precache --ios</code> only on macOS
CocoaPods issues	Outdated repos	<code>pod repo update && pod install</code> (macOS only)

Next best steps

- Add Riverpod + GoRouter and your `analysis_options.yaml`.
- Create one launch configuration per flavor in `.vscode/launch.json`:

```

1 {
2   "version": "0.2.0",
3   "configurations": [
4     {
5       "name": "Dev",
6       "request": "launch",
7       "type": "dart",
8       "program": "lib/main_dev.dart",
9       "args": ["--flavor", "dev"]
10    },
11    {
12      "name": "Prod (release)",
13      "request": "launch",
14      "type": "dart",
15      "program": "lib/main_prod.dart",
16      "flutterMode": "release",
17      "args": ["--flavor", "prod"]

```

```

18     }
19   ]
20 }

```

- Start a background codegen terminal:

```
1 dart run build_runner watch --delete-conflicting-outputs
```

- Capture perf proof early: Flutter DevTools (FPS $\geq$ 60, baseline memory $<$ 200 MB).

GitHub & CI/CD Setup Guide

Part 1: Push Your Project to GitHub

Objective: Securely version control your Flutter project and enable team collaboration through GitHub.

Step 1: Prepare Your Project for Git

```

1 # flutter create already generates a .gitignore; append the extras
2 printf '%s\n' '.env' '.env.local' '.env.production' >> .gitignore
3 printf '%s\n' '.DS_Store' 'Thumbs.db' '*.log' >> .gitignore

```

Step 2: Initialize Git and Create Initial Commit

```

1 # Initialize repository
2 git init
3
4 # Configure git (if not done globally)
5 git config user.name "Your Name"
6 git config user.email "your.email@example.com"
7
8 # Add all files and create initial commit (use multiple -m for a multiline
9   message)
10 git add .
11 git commit -m "feat: initial Flutter project setup" \
12 -m "Flutter 3.47.0 with Visual Studio Code" \
13 -m "Riverpod + GoRouter architecture" \
14 -m "Dev/Prod flavor configuration" \
15 -m "Analysis options and code quality setup"

```

Step 3: Create GitHub Repository

1. Go to <https://github.com/new>
2. **Repository name:** my-flutter-app (use kebab-case)
3. **Description:** *Flutter application with Riverpod, GoRouter, and multi-environment setup*
4. **Visibility:** Private (commercial) or Public (open source)
5. **Initialize:** *Do not* add README, .gitignore, or license
6. Click **Create repository**

Step 4: Connect and Push to GitHub

```
1 # Add the remote repository
2 git remote add origin https://github.com/your-username/my-flutter-app.git
3
4 # Rename main branch if needed (modern Git uses 'main')
5 git branch -M main
6
7 # Push with upstream tracking
8 git push -u origin main
9
10 # Verify
11 git log --oneline -5
12 git status
```

Step 5: Create Development Branch Structure

```
1 # Create and push the development branch
2 git checkout -b develop
3 git push -u origin develop
4
5 # Create a feature branch for new work
6 git checkout -b feature/add-authentication
7
8 # Standard workflow:
9 # 1) Work on feature branches
10 # 2) Merge to 'develop' for integration/testing
11 # 3) Merge 'develop' to 'main' for releases
```

This document is maintained by Dr. Mohamad AOUE
Last updated: 2026-09-20